UFUG2602: Exercise 1

Notes

- This exercise is for practice only and does not count toward the final score.
- Both 0-indexed and 1-indexed array are accepted. However, when using Python, array indices must start from 0.

Problem 1. Insertion Sort is an efficient algorithm for sorting a small number of elements. It works similarly to how people sort a hand of playing cards: each card is picked and inserted into its correct position among the already sorted cards. The algorithm iterates through the array, inserting each element into its appropriate place relative to the previous elements.

Figure 1 and Algorithm 1 demonstrate how Insertion Sort works. The index j represents the current element being inserted into the sorted portion of the array.

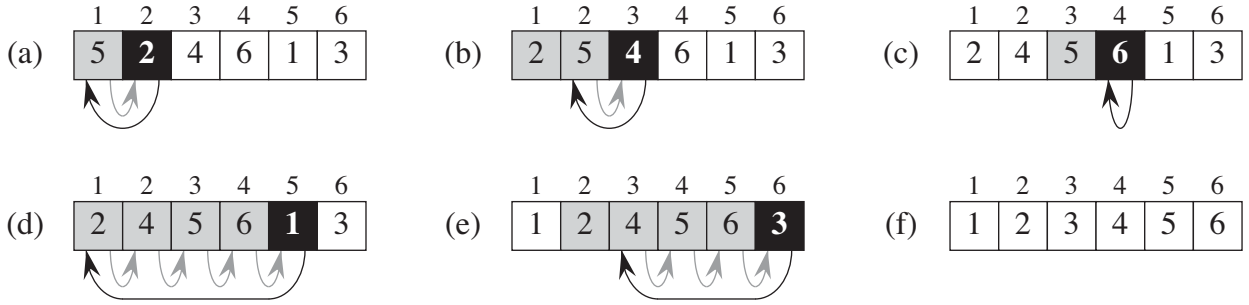


Figure 1: The operation of Insertion-Sort on the array $A = \langle 5, 2, 4, 6, 1, 3 \rangle$.

Algorithm 1 Insertion-Sort(A)

```
1: for j = 2 to A.length do
2:     key = A[j]
3:     // Insert A[j] into the sorted sequence A[1 ... j − 1].
4:     i = j − 1
5:     while i > 0 and A[i] > key do
6:         A[i + 1] = A[i]
7:         i = i − 1
8:     end while
9:     A[i + 1] = key
10: end for
```

Answer the following questions:

Q1.1. Using Figure 1 as a reference, illustrate the step-by-step execution of Insertion Sort on the array $A = \langle 31, 41, 59, 26, 41, 58 \rangle$.

Q1.2. Analyze the worst-case time complexity of the algorithm.

Q1.3. Modify Algorithm 1 to sort the array in non-increasing order instead of non-decreasing order.

Problem 2. Selection sort is a simple sorting algorithm that sorts an array by repeatedly selecting the smallest remaining element and placing it in its correct position. Given an array A of n numbers, the algorithm works as follows:
1. Find the smallest element in A and swap it with the first element, $A[1]$.
2. Find the second smallest element in the remaining unsorted part of A and swap it with $A[2]$.
3. Repeat this process for the first $n-1$ elements of A.

Q2.1. Write the pseudocode or Python implementation of the selection sort algorithm.

Q2.2. Perform a step-by-step computational cost analysis of the algorithm. Define c_i as the *cost* of step i, and determine how many *times* each step is executed. Use this analysis to derive the total running time $T(n)$.

Q2.3. Express the best-case and worst-case time complexities of selection sort using Θ-notation.

Problem 3. Binary search is an efficient algorithm for finding a target value in a sorted array by repeatedly dividing the search range in half. Given a sorted array A of size n and a target value x, answer the following questions:

Q3.1. Provide a pseudocode or Python implementation of the binary search algorithm.

Q3.2. Prove that the worst-case time complexity of binary search is $O(\log n)$.

Q3.3. Perform a step-by-step execution of the algorithm on the array $A = \langle 2, 3, 5, 7, 11, 13, 17, 19 \rangle$ when searching for $x = 17$.

Problem 4. Answer the following questions:

A	**B**	O	Ω	Θ
$5n$	n	✓	✓	✓
5	n	✓		
n^2	$2^{10}n$			
$(7/8)^n$	$(8/7)^n$			
8^n	4^n			
n^5	$32^{\log n}$			
n^{2025}	2025^n			
$n^{0.8}$	$(0.8)^n$			
$n^{\log 3}$	$3^{\log n}$			
$\log(n!)$	$\log(n^n)$			
$n^{1/\log n}$	1			
$\log^7 n$	$n^{0.7}$			
$\log \sqrt{n}$	$\sqrt[4]{n}$			
$\log(\sqrt{n})$	$\sqrt{\log n}$			

Q4.1. For each row i in the table above, determine whether A_i belongs to $O(B_i)$, $\Omega(B_i)$, or $\Theta(B_i)$. Mark all applicable classifications. No explanation is required. The first two rows are provided as examples.

Q4.2. Let $f(n)$, $g(n)$, and $h(n)$ be functions of integer n. Prove: if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

Q4.3. Prove $(2n + 2)^3 = O(n^3)$.

Q4.4. Prove $\log_2(n!) = \Theta(n \log n)$.

Problem 5. A financial analyst is monitoring stock prices over n days, where the array A represents the stock price on each day, with $A[i]$ denoting the price on day i. The analyst seeks to determine the maximum profit that can be obtained by buying and selling the stock exactly once.

Answer the following questions:

Q5.1. Provide pseudocode or Python code for an algorithm that computes the maximum possible profit from a given array A of length n.
Input: A sequence of n numbers $A = \langle a_1, a_2, \ldots, a_n \rangle$.
Output: The maximum possible profit v, defined as $v = A[j] - A[k], \quad$ where $1 \leq k < j \leq n$.

Q5.2. Analyze the time and space complexity of your algorithm.